Super-Stimulus of AI Coding Agents

Ray Feodoroff
ASCSA Member
ORCID iD: 0000-0002-5518-5570
agentorientedassurancecases@gmail.com

Abstract

The principle problem for safety assurance of any system has long been recognised as a burden, generally applied as a separate "argument", more often by a non-associative third party author, essentially tasked with making the "argument" the system is safe by way of explaining that the "correct" requirements are implemented, in a manner akin to what otherwise has the look and feel of design rationale and over the means-end decomposition of the requirements into design and implementation.

This problem of requirements as the root of assurance (safety or otherwise) seems also to have evolved as a "solution" in the AI Coding Agent space with reference to Spec Driven Development (SDD) and/or Test Driven Development (TDD) (including Behavior Driven Development or BDD).

If we have spent so much time away from writing specifications to meet the WISCY[1] urge of project management (and of customers really), and with noticeable flaws in the implementation of TDD over the decades, whether a mashup of SDD and TDD will resolve issues with "software" let alone "software safety" assurance for AI generated code begs asking.

Going back to first principles may only beg more questions, but we need to go back to first principles. Going back in time appears, therefore, a must to investigate what means are available to tackle software assurance for AI generated code.

This is otherwise a status report for an ongoing investigation towards super-stimulus for AI coding agents through a formal but approachable means to support AI coding agent driven SDD (and by extension TDD).

The goal is to support a level of software assurance, through rigor of specifications for AI coding agents by way of CRC based domain models integrated with ideas from System Theoretic Process Analysis (STPA).

Keywords: As required.

1 Introduction

This is not a proposition that AI coding agents should be involved in coding safety critical software, but the ideas behind software safety assurance are not immiscible with goals for software assurance generally and need not be immiscible with goals for assurance to be gleaned from AI generated code. Readers may, however, need moderate their definition of safety for the purposes of this discussion.

The principal problem for AI generated code is that, in any event, LLM are trained on the median for code quality, on open-source projects, projects with no formal assurance aspirations generally. That position can be "improved" by use of ad hoc subjective judgements by human trainers, with the resulting coding quality of AI coding agents still being charged with producing (versus its human counterparts) higher bug density, more vulnerabilities and greater software maintenance challenges (despite post training "tuning" approaches). Let us call that Technical Debt.

SDD and the oft cited TDD "agents" or "skills" are the current reactions to the problems for software assurance incurred by use of statistical, non-deterministic token generation, with any perception of coding "intelligence" due to the rapid generation of said tokens to entrap us with a perception of "fluency". To reign in the coding agents, principles from SDD and TDD are being applied in attempt to constrain the "context" presented to the LLM.

We fully understand that "context" is the key to directing the outputs from AI coding agents, and agents and skills interpreting SDD and TDD certainly should be encouraged as one means to constrain the context so as to narrow the outputs from the statistical, non-deterministic token generation.

Given the relatively new application of SDD in prompting coding agents, and the likely dearth of formal training in requirement engineering as a specialty, given the history of the industry, dare we envision how a formal expression of requirements may better steer a statistical yet non-deterministic process in a way requirements engineering experts from the goal oriented requirements engineering community might explain the problem, leaning on other complimentary ideas along the way. Some rigorous means to coach the specification from specification authors might, therefore, make sense.

It is as interesting then that STPA is modeling in context since Thomas's perspective of "context" and actions as requirement functions (with hazards as anti-requirements per Feodoroff (2023)). Since we could potentially re-frame SDD from a simple documentation exercise into a formal safety boundary-enforcement mechanism.

1.1 Specification as a Theory

Thomas (2013) introduces the notion of requirement as a function (context and action assigned to a controller) but Thomas discusses only ***decomposition*** of ***context*** and not of requirements, and certainly does not talk about ***refinement*** of requirements. We can address the necessity for decomposition and refinement of requirements by integrating ideas from "Problem Frames" as Feodoroff (2018, 2023) has previously recommended.

Problem Frames per Jackson (2001); Hall et al. (2010), explains the refinement of ***Specification (S)*** as being constrained both by the unyielding ***User Requirement*** (*R*), and

[1] Why Isn't Someone Coding Yet

the unyielding ***World*** (W). That is, we select for S such when we "believe" ($\Rightarrow_B$) the the expression in Equations 1 is true. The Specification S is subsequently placed upon a ***Machine*** (analogous to the "controller" of STPA).

$$W \wedge \overbrace{S}^{Machine} \Rightarrow_B \overbrace{R}^{User} \tag{1}$$

$$W = World\ full\ of\ \textbf{\textit{Phenomenon}}$$
$$S = Specification\ on\ the\ \textbf{\textit{Machine}}$$
$$R = Requirement\ function\ of\ W \times S$$

The difference between $X \vdash Y$ and $X \Rightarrow_B Y$ is if strict entailment ($\vdash$) derives Y as a theorem, then $\Rightarrow_B$ derives Y as a belief. The statement means: $W, S \Rightarrow_B R$ is an **engineering argument**, not a **mathematical proof**. There is as permissible transition point, in any event, when the specification could transition to proofs, given the work to decompose the problem into a solution is an act of concretization goals and sub-goals. Whether the specs transition to code, or to proofs then code will depend on the assurance requirements for the system under development.

Hall and Rapanotti (2008) further decorate the Problem Frame expression in terms of observations (o) and control actions (c), very much goal/agent oriented, as per Equation 2. With the same *Means-Ends* aspiration Leveson (2000) talks about in regard to Intent Specifications. We also note that S_o^c on a "machine" is syntactic sugar for the requirement function per Thomas (2013) according to Feodoroff (2023).

$$\overbrace{W \wedge S_o^c}^{Means} \Rightarrow_B \overbrace{R_{ref}^{cons}}^{Ends} \tag{2}$$

$$Desires{:}-$$
$$cons = Constrained\ Phenomenon$$
$$ref = Referenced\ Phenomenon$$
$$Intentions{:}-$$
$$c = Controlled\ Phenomenon$$
$$o = Observed\ Phenomenon$$

We can apply this to STPA based thinking since Thomas Thomas (2013) explains requirements as functions but does not formalize decomposition or refinement to the extent required for a top-down specification approach. This also allows for agent orientated style action semantics between either Jackson's *Machines* and/or STPA's *Controllers* (pick your abstraction).

In any event, given STPA is about developing beliefs about mappings between ***Context*** and ***Control Action*** we can re-write Thomas' function definition in Equation 3 - given Thomas (Thomas, 2013, p. 119) expresses requirement and hazard "functions" over the tuple of $\{SystemController(SC), ControlAction(CA), Context(Co)\}$.

$$W \wedge \overbrace{S_o^c}^{SC} \Rightarrow_B R; \quad \overbrace{o = Co, c = CA}^{\text{Thomas v Problem Frames}} \tag{3}$$

We are otherwise noting *Context* from the Cognitive Engineering ideas (cf. Leveson/Thomas and Hollnagel) is more or less as per *Phenomenon* in Problem per Jackson (2001), handy abstractions that may both have a place at differing levels of perspective of the system.

For an AI coding agent to safely leverage its generative manifold, external structures (like an STPA-derived spec framework) may then act as a topological constraint. By injecting explicit anti-requirements and behavior context and condition models into the prompt context. The developer may then force a distortion of the model's attention weights—artificially depressing the probability vectors that lead toward hazardous (in terms of anti-requirement related) regions of the manifold and forcing the stochastic sampler to remain inside safe operational bounds[2].

Per Feodoroff (2023), we can partition the "spec" space at appropriate levels of abstraciton of Problem (P), Goal (G), Requirement (R), Specification (S), and for the purpose of this paper, code (S'''). Decomposing "context" per Thomas (2013) with abstractions appropriate at each level $o \in [DomainAbstractionLevel]$, given $[P] \preccurlyeq [G] \preccurlyeq [R] \preccurlyeq [S]$:

$$\begin{aligned} &\text{P: } W_{[P]} \wedge G_{[P]} && \Rightarrow_B Solved(P); \\ &\text{G: } W_{[G]} \wedge R_{[G]} && \Rightarrow_B Realised(G); \\ &\text{R: } W_{[R]} \wedge S_{[R]} && \Rightarrow_B Operationalized(R); \\ &\text{S: } W_{[S]} \wedge S'''_{[S]} && \Rightarrow_B Encoded(S) \end{aligned} \tag{4}$$

The point being goals and requirements (and responsibilities as we will discuss later) are to be optative statements (what we want accomplished), but can still be non-operationalizable as stated (what does it mean "the system should be safe"?). Lower levels of the stack concretize the behavior with how.

We take for granted that we will often miss the $R \preccurlyeq S$ step (cf. WISCY and therefore $R \preccurlyeq S'''$) and likely can discuss vibe coding in terms of either or both $P \preccurlyeq S'''$ and or $G \preccurlyeq S'''$. In those cases we might opt into projection (potentially a delusion), in our anthropomorphism of AI, and assume AI is working over Equation 4 internally.

For mere mortal humans, it is no illusion that this is a straight forward process as Hall and Rapanotti (2008) further elaborate the problem for decomposition[3] of any one *Requirement* (R) with Equation 5, which is no different a problem for P and G and S of Equation 4.

$$\frac{W_1 \wedge S_1 \Rightarrow_B R_1 \quad ; \cdots ; \quad W_n \wedge S_n \Rightarrow_B R_n}{W \wedge S \Rightarrow_B R} \tag{5}$$

So what if assurance comes from comprehension of decomposition of intended system behavior over the entire stack represented by Equation 4. It used to.

1.1.1 Anti-Requirements and Anti-Specifications

Thomas (2013) emphasizes the need to understand the environment and context that loss (a.k.a "hazard") occurs in. Thomas calls uses the term "hazard", given the target market is safety practitioners as opposed to requirements engineers.

[2]The counter to this is that if a dangerous or buggy code pattern frequently appeared near a given prompt in training data, that path remains geometrically close and probable on the manifold - which is why CRC and TDD together make sense.

[3]Per Hall and Rapanotti (2008) the decomposition is unintitively upwards, this is not a Gentzen style proof for example.

Whereas "hazard" modeling is akin to analyzing for safety properties of formal requirements modeling.

Thomas demonstrates how to specify system behaviors resulting in both Control Actions (*CA*) and Unwanted Control Actions (*UCA*). Both *CA* and *UCA* are defined in terms akin to events driven by changes in context conditions. Liveness properties of formal requirement modeling map to the positive requirement function per Thomas (2013) and the remediation of *UCA* relates to the use of safety properties in formal requirements modeling.

Problematic is that both generalist engineers and safety practitioners tend to stay in their own lanes, so focused on "happy path" behaviors or "unhappy-path" behaviors respectively. Delivering a safety model without a "happy-path" is problematic, particularly because resolving "hazards" is a new remedial behavior governed by the correct *CA* triggered by the right context. This resolution loses meaning without the user requirements that define the intended use. There is, in any event, supposed to be a structural tension during requirements modeling between liveness and safety properties.

The approach Thomas has recommended allows for repelling from impermissible states of the system model, by control-theoretic embeddings in system design, via:

- use of context conditions triggering ***CA***; and
- asserting hazards as context conditions triggering of ***UCA*** or not triggering a required ***CA***; and
- using the antithesis of a hazard to define safety constraints which are accomplished by assigning subgoals to agents/controllers (called *Responsibilities* - $\mathbb{R}$), so as to distribute new behaviors amongst the participating controllers, designed to avoid the errant condition of the system. Really just the problem of goals versus anti-goals per Feodoroff (2023) and hence Figure 1.

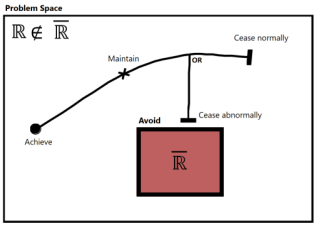


Figure 1: Avoid the anti-goal

Along with the Problem Frames analogy per Equation 3 per Feodoroff (2023) Thomas' hazard functions are, in any event, anti-requirements ($\overline{R}$) and given we can refine $\overline{R}$ into a specification we can derive anti-specification ($\overline{S}$).

1.1.2 Intentional aspects of State spaces

As suggested by Hollnagel (2025), the system's designer is to carve out intended safe specification options (we will call that set Δ), from the Universe of Discourse of infinite S (we call that Ξ). The means to describe the system as the closest approach to the safe real world is by way of use of a ***Risk Based Function*** (see Equation 6).

$$\nabla \mathscr{R}(s) \tag{6}$$

Where:

- $\mathscr{R}$ is a risk potential;
- likely candidate specifications $s \in S$, given $W, S \Rightarrow_B R$ narrows but does not close out the selection;
- gradient ∇ equals increasing or decreasing risk potential.

When specifying generally we are steering the design decisions (in the ***Direction of Maximal Loss Reduction***, safety hazards simply being one style of thinking (see Equation 7).

$$-\nabla \mathscr{H}(s) \tag{7}$$

Per Hollnagel (2025) and leveraging ideas from Thomas (2013) we can treat the control-theoretic approach as a ***State Space*** problem (see Equation 8) of ***Safe***, ***Hazardous*** and ***Unchosen*** state spaces.

$$\Sigma = \Phi \cup \Psi \cup \mathscr{U} \tag{8}$$

1. **Safe (Φ):** The user requirement space. This is the liveness target where $W \wedge S \Rightarrow_B R$. The system exists to achieve and maintain this space (we specify liveness, in terms of Φ).
2. **Hazardous (Ψ):** The unsafe space. Those S that bump the system towards the ***Direction of Maximal Hazard Introduction*** $\left(+\nabla \mathscr{H}(s)\right)$. Thomas (2013) strictly partitions this into Omission and Commission:
   (a) **Hazard of Commission ($\mathscr{C}$):** The system acts irresponsibly (c raised in known bad context Ψ):
   $$W \wedge S^{c}_{\Psi} \Rightarrow_B \overline{R}$$
   (b) **Hazard of Omission ($\mathscr{O}$):** The system fails to act responsibly when required (no c raised in known good context):
   $$W \wedge S^{\not c}_{\Phi} \Rightarrow_B \overline{R}$$
3. **Unchosen ($\mathscr{U}$):** The true orthogonal noise.

Unchosen, as a rule, is in pure user requirements terms those S that simply cannot be refined from R. That is, you accept that as long as the environment W and the S are **Good Enough** to meet the threshold of R, the design is a bounded set of possible decision points for S of $\{\Delta \subseteq \Xi\}$ (a.ka. "happy path" or the intent of the liveness properties for the system) per Equation 9.

$$\Delta = \{S \mid \exists W : (W \wedge S) \Rightarrow_B R\} \tag{9}$$

Meaning: The set of specifications S that can operationalize stakeholder intentions, in the sense that there exists a world description W for which $(W \wedge S)$ defeasibly entails the required outcomes R (remember the user's desires of Equation 2). In assurance case terms, Δ aims to bound the options that meet ***Intended Use***.

We need to further restrict options for S with the tension between functional and non-functional requirements. To this end, we recognize that rigorous safety analysis (per STPA) is simply rigorous goal/agent oriented requirements engineering analysis (we aim to avoid bad situations with safe routes around them) with the actions provided by agency (e.g., STPA for Thomas (2013) or POSE per Hall and Rapanotti (2008) or KAOS per van Lamsweerde (2009)) that aims to assert Equation 7, albeit heuristically rather than mathematically via capture of engineering arguments.

Omission, as a rule, is dealt with by its discovery via analysis and then being resolved by assigning a *CA* to an agent/controller when the associated context condition arises.

Commission, as a rule, will require an additional analysis pass, which Thomas (2013) suggests is taking the antithesis of a hazard as a safety constraint, which is an overall responsibility $\mathbb{R}$ the system should manage. We then need decompose $\mathbb{R}$ and assign the decomposed responsibilities $(\mathbb{R}_1, ..., \mathbb{R}_n)$ to agents/controllers within the system $(\mathfrak{K}_1, ..., \mathfrak{K}_n)$. Finally, refining the assigned $\mathbb{R}$ to S on the cooperating controllers.

The distribution of $\mathbb{R}_1, ..., \mathbb{R}_n$ aimed at orchestration of safety constraints across participating agents/controllers. It is otherwise up to the designer how to decide upon the means of aggregation of agent/controllers by distribution of *Context* between agent/controllers (see Figure 2). This is otherwise the basis for the control models in STPA.

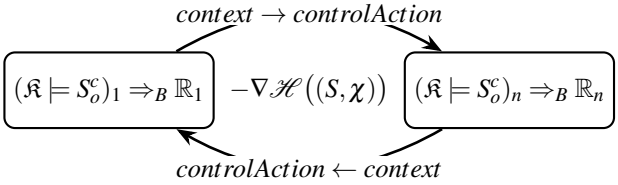


Figure 2: Refinement of $\mathbb{R}$ by trading S off with χ in the Direction of Maximal Loss over agent/controller space $\mathfrak{K}_1, ..., \mathfrak{K}_n$

Figure 2 represents the goal being to find an aggregation of state-transitions within the world state space (aka agent/controllers), over the observations of phenomenon and then reaction to that phenomenon, by use of our risk based approach that uses a lambda function of specification S traded off against a collection of system qualities χ (including safety) - qualities negotiated between the User and the Architect.

Notice, therefore, in Figure 2 we have decomposed our safety constraint $\mathbb{R}$ (the antithesis of the hazard $\overline{R}$) for the system per Equation 10. This is, in any event, goal derivation from the problem per the first layer of the stack of Equation 4.

$$\frac{\mathbb{R}_1 \ldots \mathbb{R}_n}{\mathbb{R}} \Rrightarrow_B \overline{R} \tag{10}$$

Per Xu et al. (2005) system qualities (aka non-functional requirements in χ) will be refined into operationalizable (aka functions) or checkable (observations about world phenomenon aka context). Obviously, STPA intends to turn the non-functional (so-called) system quality of safety into functional requirements to guarantee safety of the system, but we also note STPA has been used to analyze other system qualities including Security and RAM (Reliability, Availability, Maintainability) - all elements of χ.

More to the point, given inclusion of χ, we are therefore opting for assurance, by way of emphasise on requirements, of the class Firesmith (2008) recommends. Hence, Dependability rather than simply Safety assurance, meaning we are making design decisions of the form $W \wedge (S, \chi) \Rightarrow_B \mathbb{R}$, using our lambda expression for our injection of a methodical risk assessment approach from Thomas (2013) and address the recommendation of (Reinhardt, 2013, p. 56) that requirements are developed in a trustworthy manner.

1.2 Selling the past

It is little wonder software assurance is so brittle. Compare and contrast the ideas in Section 1.1 with ideas about ***Gamification of Requirements Engineering*** with essentially inadvertent goal-orientation by Lombriser et al. (2016) and still trying to discover gamification circa per Unkelos-Shpigel (2025). Which is essentially analogous to the current state of play we call vibe coding prompts.

Since the goal was to find a way to sell a formal idea by hiding it behind a comfortable paradigm already approachable by at least XP proponents we can note Evans (2004) also had proposed a rich model for assignment of $\mathbb{R}$ to the systems domain model called Class Responsibility Collaboration (CRC). CRC does not currently offer a specific means for driving towards methodically Maximal Loss Reduction - but that is only the problem of injecting the lambda function. With $\mathfrak{k}$ for Class instead of $\mathfrak{K}$ for Controller per Figure 3.

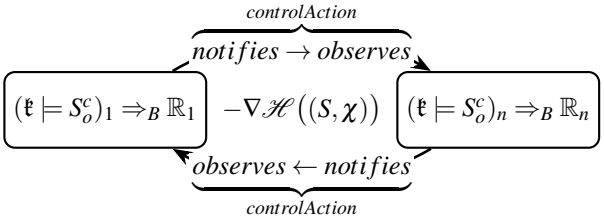


Figure 3: Refinement of $\mathbb{R}$ by trading S off with χ in the Direction of Maximal Loss over domain class space $\mathfrak{k}_1, ..., \mathfrak{k}_n$

Existing SDD tools certainly include the notion of scenario comprising GIVEN, WHEN, THEN; synonyms for context and action. Hence the CRC model can easily be exported to a format understood by current SDD tools to use the tools planning and tasking functions. With anti-requirements et al as negative guard rails.

Essentially, the just now the problem of reselling CRC. Everything old is new again.

2 Back to the Future with Industry

To bring this back to the problem of industrial practices, we can arrive at the mapping CRC and STPA paradigms as per Table 1 which warrants further investigation.

Table 1: Paradigm mapping

Intent Layer Concept	CRC	STPA
Goal allocation	Responsibility	Safety constraint
Behaviour coordination	Collaboration	Control structure
Behavioural influence	Notify	Control action
System state awareness	Shared collaboration context	Process model
Response propagation	Message passing	Feedback

2.1 Back to the Future with Domain Modeling of an eXtreme nature

Beck (1999), an advocate of XP, felt domain modeling with CRC was suitable as an input into TDD. CRC had, in fact, been introduced a decade earlier by Beck and Cunningham (1989). The main principle behind TDD, the notion of writing the test first before coding, was borrowed from ideas around left-shifting of testing, testing as early as possible, from McCracken (1957).

We cannot claim Domain Driven Design as the moniker for this approach as that is already claimed by Evans (2004). CRC plus TDD is, claimed by Beck Beck (1999), already a Domain Driven Development approach. We will call the approach CRC++.

2.2 Class Responsibility Collaboration

Class-Responsibility-Collaboration (CRC) is a domain modelling approach that obfuscates a formal semantic that has application in this context. Obfuscates since it is often portrayed as literally a card[4] based role play "game" where developers adopt the role of the classes.

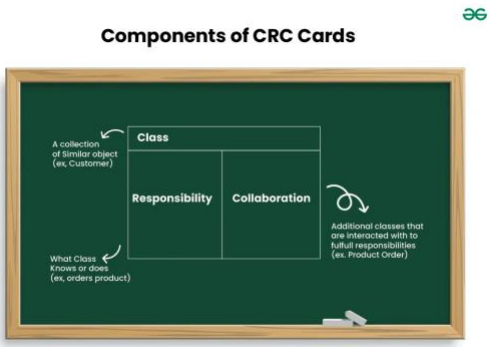


Figure 4: The "Card" Idiom

[4]https://www.geeksforgeeks.org/system-design/class-responsibility-collaboration-card/

Classes in CRC and of CRC++ are decorated with stereotypes per Table 2. STPA is therefore already accommodated as any behaviour assigned to a class stereotyped controller is inherently analogous to the notion from Thomas (2013) that a requirement is a control action raised by a controller when triggered by a context which CRC++ will accommodate as discussed below in Section 2.3. We can add additional steretypes as necessary.

Table 2: Common CRC Class Stereotypes

Stereotype	Primary role	Typical responsibilities
Entity	Represents domain concepts	Know domain state; perform domain behaviour; maintain invariants
Controller	Coordinates a use case or system operation	Receive request; coordinate objects; control workflow
Boundary	Interfaces with actors or external systems	Accept requests; present results; translate external representations

Classes are described as collections of Responsibilities ($\mathbb{R}$ per our preceding discussion). Responsibilities have categories of:

- Knowledge (state it maintains).
- Behavior (what it does).
- Constraint (rules it enforces).

Responsibilities can have collaboration modes of:

1. ***notifies:*** A decision that a class has $\mathbb{R}$ to raises a notification.
2. ***observes:*** A decision that a class has $\mathbb{R}$ to observe something;
3. ***delegatesTo:*** passing solutionizing of $\mathbb{R}$ to another class or classes;
4. ***dependsOn:*** retaining control of $\mathbb{R}$ but requiring support from another class or other classes to achieve $\mathbb{R}$.

Per Figure 3, item 1 and 2 align with the abstraction of CA per STPA and are one means to express behavioral interactions between classes.

Per item 3 and 4 above, not all $\mathbb{R}$ need operate as per Figure 3, sometimes we are just making design decisions by engineering judgment (especially those related to χ).

Responsibilities categorized as behavior but not assigned a collaboration mode can act as internalized functions within the class.

Responsibilities classified as Knowledge can, of course, define knowledge in terms of STPA-like context as there are otherwise no formalities that forbid that interpretation for responsibilities classified as knowledge.

Responsibilities classified as Constraints are interesting because setting limits or objections to outcomes is the role of anti-prompting in LLM and useful when using AI coding agents. Noting STPA hazards are in anti-requirement/specification form then we also note there are otherwise no formalities that forbid that interpretation for responsibilities classified as constraints.

2.3 Resolution of a Classical Problem in CRC (CRC++)

Traditional CRC modeling has a well-known limitation, while it is highly effective as a conceptual design and communication tool, it remains under-specified with respect to execution semantics and formal verification (see Table 3).

Table 3: CRC versus State Machines

Aspect	Domain Object Semantics (Static)	Lifecycle Semantics (Temporal)
Focus	Structure and identity of objects	Evolution and behaviour over time
Primitive	State description	State transitions
Constraint type	Invariants (always-true predicates)	Ordering and transition constraints
Logical form	Predicate logic over states	State machines / transition relations
Failure mode	Invalid or inconsistent state	Invalid or unsafe state sequence

The extension proposed here addresses this gap by introducing an explicit semantic grounding for CRC behavioral constructs, by adding the notion of Process Variable Groups to CRC classes as the means to capture states of ***Knowledge (K)***. As a result, CRC++ gains semantic closure while preserving its role as a high-level, human-readable design notation (see Table 4).

Table 4: CRC++

CRC Limitation	Semantic Resolution via STPA-Extended CRC++
Ambiguous collaborations	Mapped to guarded state transitions over *Process Variables*, providing explicit conditions under which interactions occur.
Informal responsibilities	Reinterpreted as invariants constrained by process-variable state, making obligations explicitly state-dependent.
Lack of runtime semantics	Resolved via explicit epistemic state transitions $K \rightarrow K'$, induced by observation events.
Weak causal structure	Strengthened through integration with STPA-style process-variable dependencies and control structure semantics.

3 Tool Smithing

The tool to date is being developed using AI coding agents, variously of the SDD class and those promising TDD as a "skill" applied during development – the source of affront therefore for those unkept promises and why this research is occurring.

A web-based tool was chosen given the evidence that AI coding agents cope well with boiler plating of web-based tools based upon standard frameworks, hence useability is limited to that offered by web-based. Useability is not a priority for the concept demonstrator.

A schema model for the CRC++ domain modelling has been developed iteratively. The reason for a schema is LLM based coding agents can "understand" various structured languages (xml, OWL, yaml, ...) all also suitable for translating into SDD scenarios. The CRC++ model is otherwise a knowledge graph of the specification based upon the CRC++ domain model schema.

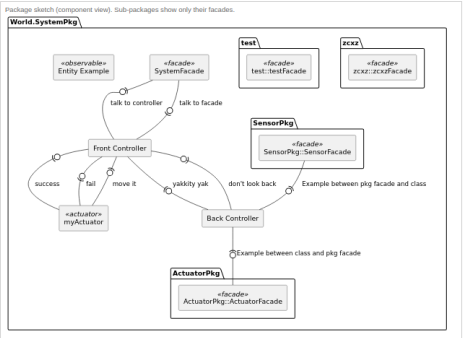


Figure 5: Package canvas

The aspiration for executable semantics, for a later iteration of the tool, would convert the specification into a form for model checkers (not for the faint hearted). A side investigation is also endearing the approach towards the specifications being executable. Neither of these avenues for model execution can be stood up without the specification tool finished.

A precis of the application, in its current state of repair follows. This is not a user manual, and the descriptions are indicative of the work to date.

3.1 Package Editing

Top-down means-ends decomposition as the basis for assurance is the claim by Leveson (2000) for Intent Specifications. Feodoroff (2023) had likened that aspiration to the decomposition "stack" over the various levels of goal decomposition per Equation 4.

To that end, a supporting design decision for the tool was to include UML style ***Package*** entities as canvases and of elements on the canvas per Figure 5. This addresses the requirement for the decomposition modeling via a ***Package Editor***.

Each *Package* will have a ***Facade*** class that acts as the boundary for the *Package*.

When a *Sub-Package* is on a canvas, only its *Facade* will be visible, you must drop into the *Package Editor* for the *Sub-Package* to see its model (per Figure 6).

CRC++ borrows somewhat from UML with the aggregation notion of ***Packages***, with the top level *Package* mandatorily the ***World***. The ***Package Canvas*** will allow either ***Class(es)*** or ***SubPackage(s)***.

Note the ***SystemPkg*** and the ***EnvironmentPkg*** are a mandatory *SubPackages* of the ***WorldPkg*** package, with the *WorldPkg* package created mandatorily when a project is created. Below that is the designers prerogative.

It is really up to the designer as to where *G*, *R* and *S* sit at, with system goals partitioned into responsibilities ($\mathbb{R}$) across agent/controllers and with phenomenological aspects of *R* and *S* decomposed by the designer dependent on the problem being framed per Hall and Rapanotti (2008); Hall et al. (2010); Jackson (2001).

The Package Editor is where the STPA-like Control Di-

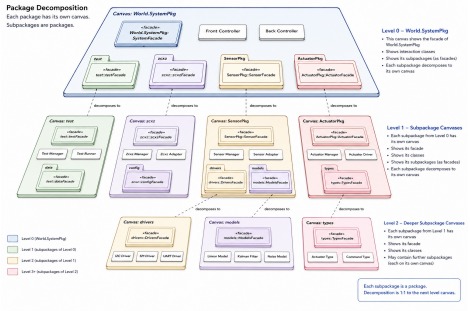


Figure 6: Decomposition by SubPackage

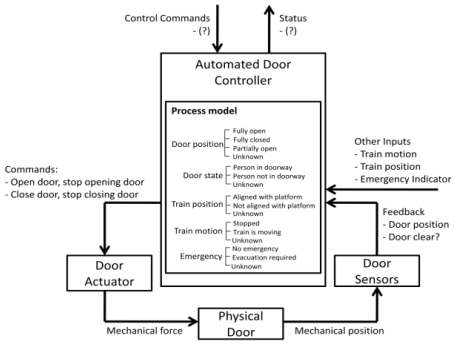


Figure 7: Process Model (Thomas, 2013, p. 82)

agrams are authored, essentially defining the Notifies and Observes Collaborations of the CRC idiom, accessible by an Actions (Editor) button. A Class and Sub-Packages button provides access to Class and (Sub)Package editors.

In support of argentic reasoning, the Package Editor does force the $notifies \rightarrow observes$ relationships as they are added and then automatically sets up notifies and observes responsibilities on collaborating the classes.

As one would expect, the visual style helps discover Omission ($\mathcal{O}$) of CA when modeling in the requirements phase before "hazard" analysis.

The Package Canvas should be familiar to STPA proponents as the (so-called) control diagram. It will be as familiar to old guard software developers as a Control Flow Diagram (CFD) - given notifications are events currently without payloads.

3.2 Class Editing

The Class Editor includes access to the Process Variable Groups Editor, the Responsibilities Editor and of setting Class meta data.

The editors all have the same look and feel, based upon idioms from the webapp service.

Figure 8 depicts the landing page for the Class Editor. Notice View and Edit buttons both for the companion Responsibility viewing/editing and the Process Variable Group viewing/editing.

Note also the "Class type" stereotype radio buttons for setting the Class role. The "boundary" role can be further decorated to "sensor" or "actuator" to support STPA idioms.

3.2.1 Process Variable Groups

Per Figure 7, STPA requires development of a process model based upon context variables according to Thomas (2013). To wit, the Class Editor includes "Process Variable Groups" to allow for multiple groups of context variables and their enumerated optional variables to be viewed and edited (Figure 8 and Figure 9 respectively).

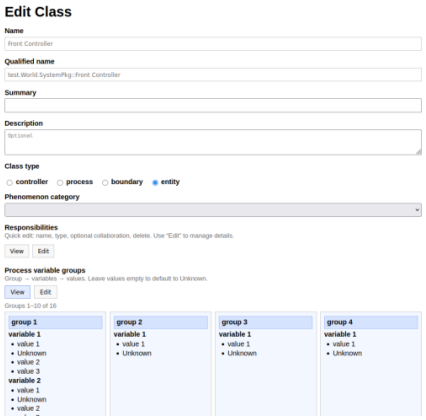


Figure 8: Process Variable View

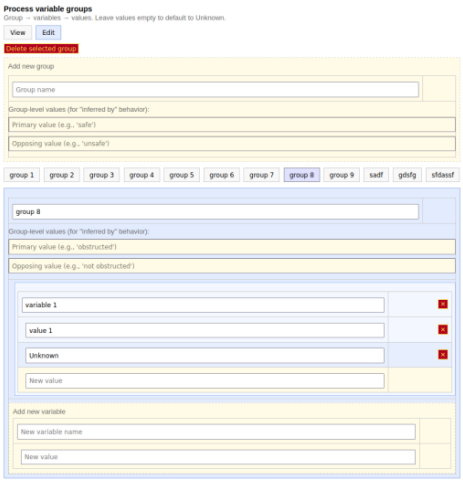


Figure 9: Process Variable Edit

3.2.2 Responsibility Editor

Recalling that responsibilities are modal, and include specification of knowledge, behaviour and constraints.

Notifies

The more interesting editor is the Notifies Responsibility since this is analogous to the SpecTRM-RL tables (see Figure 10). This is simply a table of AND/OR miniterms of the ilk: IF minterm(s) THEN CA - and hence behavioral.

Of particular note is the idiom that the first miniterm

column is the user requirement column, and subsequent columns are additional requirements discovered during "hazard" analysis.

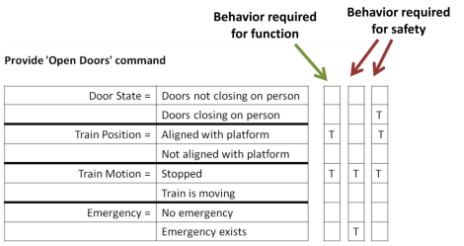


Figure 10: Example SpecTRM-RL table (Thomas, 2013, p. 121)

It is somewhat circumstantial but Figure 10 is reflective of the discussion under Section 1.1.2 re the reflection on definition of requirements via CA and then the two cuts of anti-requirements needing to be resolved by introduction of remedial contexts driving CA (first by addressing $\mathscr{O}$ and then by addressing $\mathscr{C}$).

The commensurate view and editor pages from CRC++ are Figure 11 and Figure 12 respectfully. Note the DNF (Disjunctive Normal Form) expression of the miniterms, suitable for prompt injection.

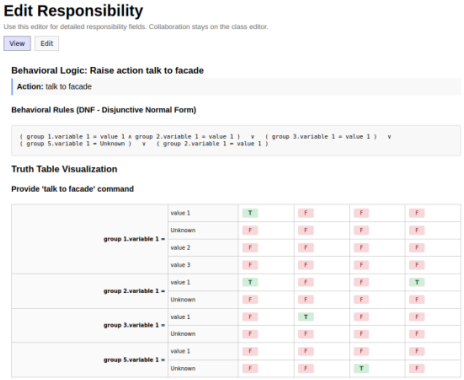


Figure 11: Notifies View

Thomas (2013) explains this table *ProcessVariables* → *ControlAction*, hence we wish to trigger on a knowledge clause defined by a particular context defined by a miniterm of context variable states, hence ***IfKnowledgeClause(IKC)*** of Equation 11.

$$\text{IF } IKC \text{ THEN } controlAction \tag{11}$$

$$\text{IKC} \equiv \bigvee_{i=1}^{N} \left(\bigwedge_{j=1}^{M} (G.var_j \in V_{i,j}) \right) \tag{12}$$

Where N is the number of decision columns, M is the number of process variables, and $V_{i,j} \subseteq \text{Dom}(G.var_j)$ denotes the set of satisfying values marked for variable j in column i (with $V_{i,j} = \text{Dom}(G.var_j)$ when no value is specified).

This can also be usefully turned into a ***UpdateKnowledgeClause(UKC)*** , representing $K \rightarrow K^{'}$ on the next clock ($G.var \rightarrow G.var^{'}$):

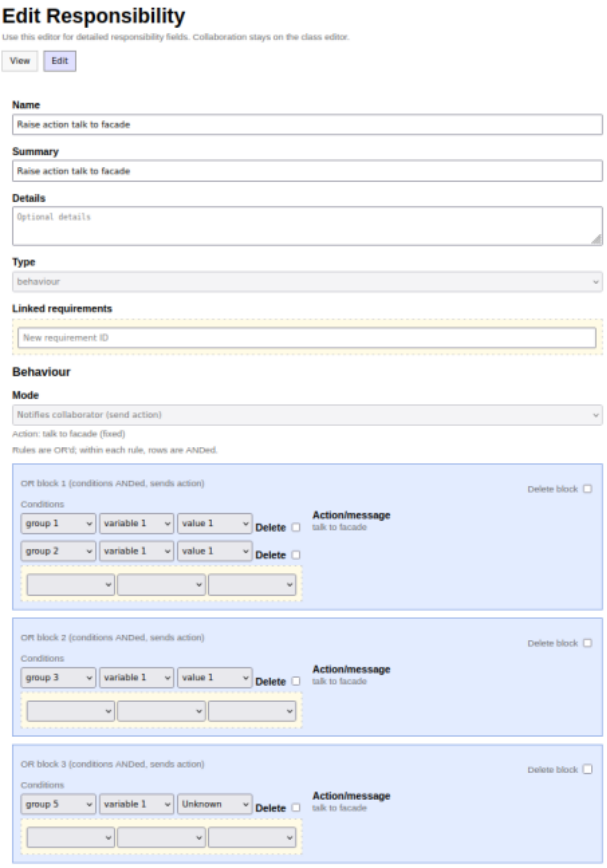


Figure 12: Notifies Edit

$$\text{UKC} \equiv \bigvee_{i=1}^{N} \left(\bigwedge_{j=1}^{M} (G.var_j \in V_{i,j}) \implies \bigparallel_{j=1}^{M} (G.var'_j \leftarrow val_{i,j}) \right) \tag{13}$$

Where N is the number of rules, M is the number of variables, $V_{i,j} \subseteq \text{Dom}(G.var_j)$ represents the valid trigger values, and $G.var'_j$ denotes the next-state value of variable j.

Observes

We want UKC since we can then introduce executable semantics to therefore provide requirement validation opportunities when using SDD style approaches when using AI coding agents.

For example, the behavioral responsibility called observes. This is a departure from what Thomas (2013) describes as *observes* provides a gated response to an incoming notification. Figure 13 depicts the *observes* editor[5]. The logic of *observes* being of the form in Equation 14.

[5]Space does not permit depiction of the commensurate *observes* view. However all behaviour views are of the same ilk as the *notifies* view, with rendered DNF equations for embedding in prompting.

$$observes \xrightarrow{\text{gate}} action \tag{14}$$

$$\text{where } gate = \begin{cases} NULL \\ IKC \end{cases} \quad , \quad action = \begin{cases} NULL \\ UKC \end{cases}$$

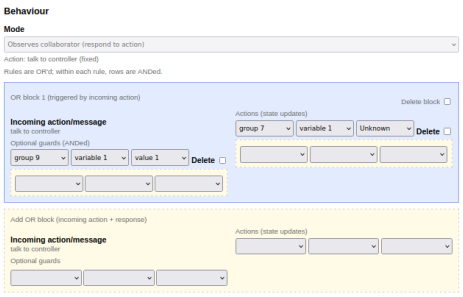


Figure 13: Observation Edit

The presumption here is guarding on transitions is a natural instrument of state machines when specifying requirements. Discovery of UCA of type commission (raised by the agent/controller that *notifies*) may therefore be dealt with by application of a guard on the agent/controller that *observes*. It meets the intent within STPA of remediation by distribution of sub-goals of type $\mathbb{R}$.

Behavior

The responsibility mode of *behavior* (internal) (a.k.a $K \rightarrow K^{'}$) is of Equation 15. This allow for a knock on behavior change when an *observes* event changes context variables. It analogous to an internal function.

$$\text{IF } IKC \text{ THEN } UKC \tag{15}$$

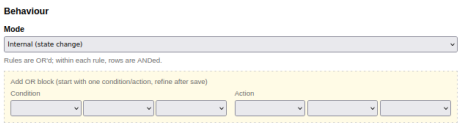


Figure 14: Internal Behavior Edit

3.3 Other Editors

The remaining editors needing to be developed to support the "hazard" analysis aspect are around context inference and hazardous control judgments to be made by the analyst. They are last to be developed since they are based upon the *CA* model embedded in the CRC++ model.

Infers

(Thomas, 2013, p. 138) recommends that the state of process model variables may need be inferred from various inputs, see Figure 15. This is akin to the problem of DNF miniterm assignment generally, hence is in the development plan for the CRC++ application.

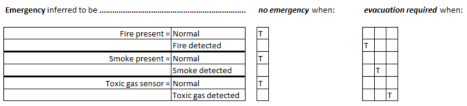


Figure 15: Inference Table (Thomas, 2013, p. 140)

The Inference table then closes the loop on the Context Table generation, see Figure 16 . The Context Table includes inferred conditions such as "Emergency" per Figure 7 and Figure 10. Albiet in the CRC++ version two user response columns will be provided, one for $\mathscr{O}$ and a second for $\mathscr{C}$ (Figure 16 only includes $\mathscr{O}$ and we note the variation of format of context tables across various pubs).

Control Action	Train Motion	Emergency	Train Position	Door State	Hazardous if not provided in this context?
Door open command not provided	Train is stopped	No emergency	Aligned with platform	Person not in doorway	No[26]
Door open command not provided	Train is stopped	No emergency	Not aligned with platform	Person not in doorway	No
Door open command not provided	Train is stopped	No emergency	Aligned with platform	Person in doorway	Yes
Door open command not provided	Train is stopped	No emergency	Not aligned with platform	Person in doorway	No[27]
Door open command not provided	Train is stopped	Emergency exists	(doesn't matter)	(doesn't matter)	Yes
Door open command not provided	Train is moving	(doesn't matter)	(doesn't matter)	(doesn't matter)	No

Figure 16: Context Table (Thomas, 2013, p. 133)

The Context Table is apt to act in place of the *Constraint* mode of responsibilities with the CRC++ model - essentially to define the negative prompting "IF hazard THEN avoid" which might leave the AI coding agent to contrive remediation approaches.

Hazardous Control Actions

The classic SPTA Hazardous Action Table (17) will be created from the *CA* embedded in the CRC++ model so as to allow for capture of decisions by the analyst as to which contexts lead to hazards due to behavioral errors in the system - to be further specified in the *Context* tables.

Control Action	Hazardous Control Actions			
	Not Providing Causes Hazard	Providing Causes Hazard	Wrong Timing or Order Causes Hazard	Stopped Too Soon or Applied Too Long

Figure 17: Hazardous Control Actions Table

3.4 Other functions coming

At a minimum, the tool is already generating a CRC model that an XP proponent might recognize.

The internal structured representation of the CRC++ model can readily be exported to formats suitable for SDD tools such as Google's Spec Kit and the competing OpenSpec agent. The "scenario" descriptions of the SDD approaches akin to scenario descriptions in BDD tools especially.

Test generation using the CRC++ domain model can target various TDD and BDD frameworks to circumvent the generation of text cases via the AI coding agent to provide some assurance the test cases relate the specification model, without an intermediate stochastic but non-deterministic token generation step.

An MCP service will be developed to allow for a coding agent to interrogate the domain model (much like MCP servers that provide knowledge graphs).

The agentic specification style is also suitable for describing agent, including AI agent, interactions; at a level of abstraction suitable for prompting.

As discussed, the candidate for requirement validation modeling, via executable semantics, has static elements in the domain model, awaiting a means to orchestrate the abstracted behavior. The semantics of the domain model are aimed, otherwise, to experiment with exporting into a format for model checking.

4 CONCLUSIONS

The proposed CRC++ model serves as a "friendly face" to bridge the gap between high-assurance formalisms and the familiar idioms of Agile development.

We propose transforming CRC into CRC++ by:

- Resolving ambiguous collaborations are into guarded state transitions, providing explicit conditions for when interactions between entities occur.
- Preserving the formal rigor of Linear Temporal Logic (LTL) "hidden" within these executable models.
- Providing an architecture that ensures that the specification remains reality-anchored by clustering entities into cohesive, structured collection of integrity units (Aggregates) that correspond directly to the cohesive concepts discovered during analysis.

By integrating process variables and state-transition modeling into the traditional Class-Responsibility-Collaboration (CRC) framework, the "symbol soup" of formal Requirements Engineering "science" is embedded directly into the design process.

While TDD is a staple of eXtreme Programming (XP), it often lacks formal domain modeling as an input. By re-adopting CRC as the domain modeling layer, developers can provide AI agents with the necessary logical integrity boundaries and tactical rules needed to generate code that is accurately yoked to the originating requirements.

The "Symbol Soup" Behind the "Friendly Face" The complexity of the paper's earlier sections is the "symbol soup" of control theoretics and logic has been shown to be the underlying semantic engine for CRC++.

5 References

Beck, K. (1999). *Extreme Programming Explained: Embrace Change*. Addison-Wesley Professional, Boston, MA.

Beck, K. and Cunningham, W. (1989). A laboratory for teaching object-oriented thinking. In *Proceedings of the Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*, pages 1–6. ACM.

Evans, E. (2004). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley Professional.

Feodoroff, R. (2018). Back to the future: Pollock versus toulmin. In *2018 ASSC Proceedings*.

Feodoroff, R. (2023). Human orientedness of system safety. In *2023 ASSC Proceedings*.

Firesmith, D. (2008). Quality assessment of system architectures and their requirements (quasar) version 3.0. Software Engineering Institute, Carnegie Mellon University, Presentation. Version 3.0, February 13, 2008.

Hall, J. G. and Rapanotti, L. (2008). Assurance-driven design. In *2008 The Third International Conference on Software Engineering Advances*, pages 379–388.

Hall, J. G., Rapanotti, L., and Jackson, M. (2010). Problem oriented software engineering. Technical Report 2010/03, Department of Computing, The Open University, Walton Hall, Milton Keynes, United Kingdom.

Hollnagel, E. (2025). The folly of safety-III. `https://erikhollnagel.com/onewebmedia/Folly%20counterarguments%20mar%2015.pdf`. Accessed: 2026-05-16.

Jackson, M. A. (2001). *Problem Frames: Analysing and Structuring Software Development Problems*. Addison-Wesley Longman Publishing Co., Inc., USA.

Leveson, N. (2000). Intent specifications: an approach to building human-centered specifications. *IEEE Transactions on Software Engineering*, 26(1):15–35.

Lombriser, P., Dalpiaz, F., Lucassen, G., and Brinkkemper, S. (2016). Gamified requirements engineering: Model and experimentation. In *Requirements Engineering: Foundation for Software Quality (REFSQ 2016)*, volume 9619 of *Lecture Notes in Computer Science*, pages 171–187. Springer.

McCracken, D. D. (1957). *Digital Computer Programming*. John Wiley & Sons, Inc., New York, NY.

Reinhardt, D. W. (2013). *Safety Assurance of Aviation Systems*. PhD thesis, University of York. Department of Computer Science.

Thomas, J. (2013). Extending and automating a systems-theoretic hazard analysis for requirements generation and analysis.

Unkelos-Shpigel, N. (2025). Elicit, specify, require, revise: enhancing requirements engineering process using gamification. *Software Quality Journal*, 33(3).

van Lamsweerde, A. (2009). *Requirements engineering*. John Wiley & Sons, Chichester, England.

Xu, L., Ziv, H., Alspaugh, T. A., and Richardson, D. J. (2005). An architectural pattern for non-functional dependability requirements. *ACM SIGSOFT Software Engineering Notes*, 30:1 – 6.